

# **Tugas 4 - II4021 Kriptografi**

## **Implementasi Shamir Secret Sharing pada Aplikasi Password Manager Terdistribusi**

Diampu oleh:  
**Prof. Dr. Ir. Rinaldi, M.T.**



Disusun oleh :

Khairunnisa Azizah                      18223117

Leonard Arif Sutiono                      18223120

Nakeisha Valya Shakila                      18223133

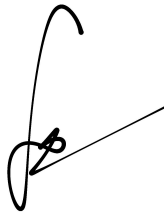
**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI**  
**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**  
**INSTITUT TEKNOLOGI BANDUNG**  
**JATINANGOR**  
**2026**

## LEMBAR PERNYATAAN

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (raw output) dari Generative AI, melainkan hasil pengembangan dan penulisan mandiri.



Khairunnisa Azizah



Leonard Arif Sutiono



Nakeisha Valya Shakila

# DAFTAR ISI

|                                                                             |          |
|-----------------------------------------------------------------------------|----------|
| <b>LEMBAR PERNYATAAN.....</b>                                               | <b>2</b> |
| <b>DAFTAR ISI.....</b>                                                      | <b>3</b> |
| <b>DAFTAR GAMBAR.....</b>                                                   | <b>5</b> |
| <b>DAFTAR TABEL.....</b>                                                    | <b>6</b> |
| <b>BAB I</b>                                                                |          |
| <b>PENDAHULUAN.....</b>                                                     | <b>7</b> |
| 1.1. Latar Belakang.....                                                    | 7        |
| 1.2. Rumusan Masalah.....                                                   | 7        |
| 1.3. Tujuan.....                                                            | 8        |
| <b>BAB II</b>                                                               |          |
| <b>PEMBAHASAN.....</b>                                                      | <b>9</b> |
| 2.1. Dasar Teori.....                                                       | 9        |
| 2.1.1. Shamir Secret Sharing (SSS).....                                     | 9        |
| 2.1.2. AES-128-GCM.....                                                     | 10       |
| 2.1.3. Key Derivation Function (KDF).....                                   | 11       |
| 2.1.4. Cryptographically Secure Pseudorandom Number Generator (CSPRNG)..... | 12       |
| 2.1.5. SQLite dan Arsitektur Client-Server.....                             | 13       |
| 2.2. Perancangan Sistem.....                                                | 14       |
| 2.2.1. Arsitektur dan Penyimpanan Data.....                                 | 14       |
| 2.2.2. Perancangan Alur Operasional Sistem.....                             | 16       |
| 2.2.3. Perancangan Modul Sistem.....                                        | 19       |
| 2.3. Implementasi Sistem.....                                               | 23       |
| 2.3.1. Implementasi Shamir Secret Sharing.....                              | 23       |
| 2.3.2. Implementasi KDF dan Proteksi Local Share.....                       | 24       |
| 2.3.3. Implementasi AES-128-GCM pada Vault.....                             | 25       |
| 2.3.4. Implementasi Penyimpanan Data dan Server.....                        | 26       |
| 2.3.5. Implementasi Pengelolaan Password.....                               | 28       |
| 2.3.6. Implementasi Password Generator dan Mode Backup.....                 | 29       |
| 2.4. Pengujian dan Analisis Hasil.....                                      | 30       |
| 2.4.1. Menyalakan Server.....                                               | 30       |
| 2.4.2. Pengujian Pembuatan Vault.....                                       | 31       |
| 2.4.3. Pengujian Penyimpanan dan Perlindungan Local Share.....              | 31       |
| 2.4.4. Pengujian Akses Normal.....                                          | 31       |
| 2.4.5. Pengujian Penambahan Data Password.....                              | 32       |
| 2.4.6. Pengujian Pengubahan dan Penghapusan Data Password.....              | 32       |
| 2.4.7. Pengujian Penyimpanan Vault pada Server.....                         | 32       |
| 2.4.8. Pengujian Mode Backup.....                                           | 33       |
| 2.4.9. Pengujian Kegagalan Pemulihan.....                                   | 33       |

|                                                            |           |
|------------------------------------------------------------|-----------|
| 2.4.10. Pengujian Bonus Kriptografi Visual (Opsional)..... | 33        |
| <b>BAB III</b>                                             |           |
| <b>PENUTUP.....</b>                                        | <b>36</b> |
| 3.1. Kesimpulan.....                                       | 36        |
| 3.2. Saran.....                                            | 37        |
| 3.3. Lampiran.....                                         | 38        |



## DAFTAR GAMBAR

|                                                                                               |    |
|-----------------------------------------------------------------------------------------------|----|
| Gambar 2.1 Diagram Komponen dan Hubungan Antar-Modul Aplikasi.....                            | 14 |
| Gambar 2.2 Sequence Diagram Perancangan Alur Pembuatan Vault.....                             | 17 |
| Gambar 2.3 Flow Diagram Akses Normal Sistem.....                                              | 18 |
| Gambar 2.4 Flow Diagram Penanganan Kegagalan Rekonstruksi Rahasia.....                        | 18 |
| Gambar 2.5 Arcitecture Diagram Modul Sistem.....                                              | 21 |
| Gambar 2.6 Diagram Implementasi Shamir Secret Sharing.....                                    | 23 |
| Gambar 2.7 Diagram Implementasi KDF dan Proteksi Local Share.....                             | 25 |
| Gambar 2.8 Diagram Penyimpanan Data dan Server.....                                           | 26 |
| Gambar 2.9 Diagram Implementasi Penyimpanan Data dan Sinkronisasi Server.....                 | 27 |
| Gambar 2.10 Diagram Alur Implementasi Pengelolaan dan Pengamanan Password.....                | 28 |
| Gambar 2.11 Password Generator dan Mode Backup.....                                           | 30 |
| Gambar 2.12 Output Berhasil Menyalakan Server.....                                            | 31 |
| Gambar 2.13 Output Berhasil Membuat Vault.....                                                | 31 |
| Gambar 2.14 Penyimpanan dan Perlindungan Local Share Dengan Enkripsi.....                     | 31 |
| Gambar 2.15 Output Berhasil Mengakses Vault dengan.....                                       | 31 |
| Gambar 2.16 Output Berhasil Menambahkan Data Password.....                                    | 32 |
| Gambar 2.17 Output Berhasil Melakukan Perubahan Data Password.....                            | 32 |
| Gambar 2.18 Output Berhasil Melakukan Penghapusan Data Password.....                          | 32 |
| Gambar 2.19 Penyimpanan Vault pada Server (Database SQLite).....                              | 33 |
| Gambar 2.20 Mematikan Server.....                                                             | 33 |
| Gambar 2.21 Output Berhasil Melakukan Recovery dengan Menggunakan Recovery Share.....         | 33 |
| Gambar 2.21 Output Gagal Melakukan Recovery dengan menggunakan Recovery Share yang Salah..... | 33 |
| Gambar 2.22 Output Berhasil Membuat QR.....                                                   | 34 |
| Gambar 2.23 Tampilan QR.....                                                                  | 34 |
| Gambar 2.24 Hasil Scan QR Menampilkan Recovery Share.....                                     | 35 |

## DAFTAR TABEL

|                                                                |    |
|----------------------------------------------------------------|----|
| Tabel 2.1 Pembagian Data pada Lokasi Penyimpanan Sistem.....   | 15 |
| Tabel 2.2 Pemetaan Fungsi dan Tanggung Jawab Modul Sistem..... | 21 |

# BAB I

## PENDAHULUAN

### 1.1. Latar Belakang

Perkembangan teknologi digital menyebabkan pengguna memiliki banyak akun pada berbagai layanan, seperti email, media sosial, dan portal akademik. Setiap akun umumnya menggunakan password yang berbeda demi menjaga keamanan data. Namun, banyaknya password membuat pengguna kesulitan mengingat dan mengelolanya secara aman. Penggunaan password yang sama pada banyak akun juga meningkatkan risiko kebocoran data apabila salah satu akun berhasil diretas.

Untuk mengatasi masalah tersebut, digunakan aplikasi *password manager* sebagai tempat penyimpanan data akun secara terpusat. Meskipun demikian, penyimpanan seluruh data pada satu sistem tetap memiliki risiko keamanan apabila server atau *master key* berhasil diakses oleh pihak yang tidak berwenang. Oleh karena itu, diperlukan mekanisme tambahan untuk meningkatkan keamanan penyimpanan password.

Pada tugas ini, dikembangkan aplikasi *password manager* terdistribusi yang menerapkan *Shamir Secret Sharing* dengan skema (2,3) untuk membagi *master key* menjadi beberapa *share*. Selain itu, sistem menggunakan AES-128-GCM untuk enkripsi vault, *Key Derivation Function* (KDF) untuk melindungi *local share*, dan *Cryptographically Secure Pseudorandom Number Generator* (CSPRNG) untuk membangkitkan password otomatis. Dengan pendekatan ini, sistem diharapkan mampu meningkatkan keamanan penyimpanan password sekaligus menyediakan mekanisme pemulihan data ketika server tidak dapat diakses.

### 1.2. Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, rumusan masalah dalam tugas ini adalah sebagai berikut:

1. Bagaimana cara mengimplementasikan aplikasi *password manager* terdistribusi dengan arsitektur *client-server*?
2. Bagaimana penerapan *Shamir Secret Sharing* dengan skema ambang batas (2,3) untuk melindungi *master key*?

3. Bagaimana proses enkripsi dan dekripsi vault menggunakan AES-128-GCM agar data password tetap aman?
4. Bagaimana mekanisme penyimpanan *local share*, *server share*, dan *recovery share* agar tidak ada pihak yang menyimpan *master key* secara utuh?
5. Bagaimana implementasi mode normal dan mode backup untuk menjaga akses terhadap vault ketika server tidak dapat diakses?
6. Bagaimana penerapan CSPRNG dalam proses pembangkitan password otomatis yang aman?

### 1.3. Tujuan

Berdasarkan latar belakang yang telah diuraikan, maka terdapat beberapa tujuan yang ingin dicapai dalam pengerjaan tugas ini, yaitu sebagai berikut:

1. Mengembangkan aplikasi *password manager* terdistribusi berbasis *client-server*.
2. Mengimplementasikan *Shamir Secret Sharing* dengan skema (2,3) untuk membagi dan merekonstruksi *master key*.
3. Mengimplementasikan AES-128-GCM untuk menjaga kerahasiaan dan integritas data pada vault.
4. Mengimplementasikan mekanisme perlindungan *share* dan penyimpanan data terenkripsi sesuai prinsip *zero-knowledge*.
5. Mengimplementasikan mode normal dan mode backup sebagai mekanisme akses dan pemulihan vault.
6. Mengimplementasikan fitur pembangkitan password otomatis menggunakan CSPRNG.
7. Menganalisis keamanan dan fungsionalitas sistem melalui serangkaian pengujian implementasi.

## BAB II

### PEMBAHASAN

#### 2.1. Dasar Teori

##### 2.1.1. Shamir Secret Sharing (SSS)

*Shamir's Secret Sharing* (SSS) merupakan sebuah metode kriptografi yang dirancang oleh Adi Shamir untuk membagi sebuah data rahasia (*secret*) menjadi beberapa bagian terpisah yang disebut *shares*. Prinsip utama dari skema ini adalah memecah informasi sedemikian rupa sehingga rahasia asli tidak dapat diprediksi sama sekali jika hanya mengandalkan satu atau beberapa *shares* saja. Rahasia tersebut hanya dapat dikonstruksi ulang apabila sejumlah *shares* dengan batas minimum tertentu berhasil dikumpulkan dan digabungkan kembali. Mekanisme kerja dari S ini bersandar pada konsep matematika yang kuat, dengan rincian implementasi sebagai berikut:

- Konsep Skema Threshold (k,n): Berdasarkan metode yang diperkenalkan oleh Adi Shamir, skema threshold (t,n)), yang dalam beberapa literatur juga dituliskan sebagai (k,n), merupakan metode pembagian rahasia S kepada n partisipan. Skema ini menjamin bahwa setiap himpunan yang terdiri atas minimal t partisipan dapat merekonstruksi rahasia S. Sebaliknya, apabila jumlah *share* yang tersedia kurang dari t (maksimal t-1 *share*), maka tidak ada informasi apa pun mengenai rahasia S yang dapat diperoleh, sehingga rahasia tetap tidak dapat ditentukan (*completely undetermined*).
- Mekanisme Pembentukan Share: Proses pembangkitan *share* dilakukan oleh *dealer* dengan memanfaatkan konsep polinomial dan aritmetika modular. Langkah pertama adalah memilih sebuah bilangan prima p yang lebih besar daripada nilai rahasia S dan jumlah partisipan n ( $p > S$ ). Selanjutnya, dipilih t-1 bilangan bulat acak dalam modulo p, yaitu  $a_1, a_2, \dots, a_{t-1}$ , untuk membentuk polinomial berderajat t-1 sebagai berikut:

$$f(x) \equiv S + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1} \pmod{p}$$

Fungsi polinomial tersebut dikonstruksikan sedemikian rupa sehingga nilai konstanta awalnya merepresentasikan rahasia itu sendiri, yaitu  $f(0) \equiv S \pmod{p}$ . Setiap partisipan ke-i kemudian diberikan sepasang koordinat unik ( $x_i, y_i$ ) sebagai *share*, dengan  $x_i$

merupakan nilai pembeda yang umumnya dipilih secara berurutan ( $x_1 = 1, x_2 = 2, \dots, x_n = n$ ), sedangkan  $y_i$  diperoleh dari hasil evaluasi fungsi polinomial pada titik tersebut, yaitu  $y_i \equiv f(x_i) \pmod{p}$ . Untuk menjaga keamanan sistem, bentuk polinomial  $f(x)$  harus dirahasiakan, sementara nilai bilangan prima  $p$  dapat diketahui secara publik.

- Mekanisme Rekonstruksi Secret: Ketika sedikitnya  $t$  partisipan berkumpul untuk memulihkan rahasia  $S$ , mereka dapat menggunakan koordinat  $(x_k, y_k)$  yang dimiliki untuk merekonstruksi polinomial semula. Proses ini dapat dilakukan melalui dua pendekatan matematis. Pendekatan pertama adalah membentuk Sistem Persamaan Linear (SPL) yang memuat variabel  $S, a_1, \dots, a_{t-1}$  dalam modulo  $p$ , kemudian menyelesaikannya menggunakan metode eliminasi Gauss atau Gauss-Jordan. Pendekatan kedua adalah menggunakan metode Interpolasi Lagrange. Melalui polinom Lagrange berderajat  $t-1$  dalam modulo  $p$ , fungsi polinomial dapat direkonstruksi dari  $t$  titik yang tersedia menggunakan persamaan:

$$p(x) \equiv y_1 L_1(x_1) + y_2 L_2(x_2) + \dots + y_t L_t(x_t) \pmod{p}$$

yang dalam hal ini: 
$$L_k(x) = \prod_{\substack{i=1 \\ i \neq k}}^t \frac{x - x_i}{x_k - x_i} \quad k = 1, 2, \dots, t$$

Setelah polinomial berhasil direkonstruksi, nilai rahasia asli dapat diperoleh dengan mengevaluasi fungsi pada  $x = 0$ , sehingga diperoleh  $S = p(0)$ .

Karena sebuah polinomial berderajat  $t-1$  hanya dapat ditentukan secara unik oleh minimal  $t$  titik berbeda, maka partisipan yang memiliki kurang dari  $t$  *share* tidak akan mampu merekonstruksi polinomial maupun memperoleh informasi apa pun mengenai rahasia yang disimpan.

### 2.1.2. AES-128-GCM

Advanced Encryption Standard (AES) merupakan algoritma kriptografi kunci simetris yang banyak digunakan sebagai standar global untuk mengamankan data. Pada AES-128, angka 128 menunjukkan panjang kunci yang digunakan, yaitu 128 bit. Data dienkripsi dalam blok-blok berukuran tetap melalui beberapa tahapan transformasi yang dirancang agar sulit dipecahkan tanpa kunci yang benar. Dalam implementasinya, AES sering dipadukan dengan berbagai mode operasi, salah satunya adalah *Galois/Counter Mode* (GCM), untuk memberikan tingkat keamanan yang lebih baik. Integrasi AES dengan GCM

menghasilkan algoritma AES-128-GCM yang tidak hanya mengenkripsi data, tetapi juga memastikan keaslian dan keutuhan data tersebut. Beberapa karakteristik penting dari AES-128-GCM adalah sebagai berikut:

- Mode Operasi GCM: GCM merupakan mode *Authenticated Encryption with Associated Data* (AEAD), yaitu metode yang menggabungkan proses enkripsi dan autentikasi dalam satu mekanisme. Dengan demikian, data tidak hanya disembunyikan dari pihak yang tidak berwenang, tetapi juga dapat diverifikasi keasliannya setelah diterima.
- Kerahasiaan dan Integritas Data: AES-GCM memberikan dua lapisan keamanan sekaligus. Kerahasiaan (*confidentiality*) dijaga melalui proses enkripsi AES sehingga isi data tidak dapat dibaca tanpa kunci yang sesuai. Sementara itu, integritas (*integrity*) dijaga melalui *authentication tag*, yaitu kode autentikasi yang digunakan untuk memastikan bahwa data tidak mengalami perubahan, manipulasi, atau kerusakan selama proses penyimpanan maupun pengiriman.
- Penggunaan Nonce: Pada AES-GCM, setiap proses enkripsi harus menggunakan *nonce* (*number used once*), yaitu nilai unik yang hanya boleh digunakan satu kali untuk setiap kunci. Penggunaan *nonce* yang sama secara berulang pada kunci yang sama dapat menimbulkan risiko keamanan yang serius karena berpotensi menyebabkan kebocoran informasi dan mengurangi keandalan mekanisme autentikasi.

AES-128-GCM dipilih dalam banyak aplikasi keamanan data karena memiliki performa yang cepat dan efisien. Mode GCM juga memungkinkan proses enkripsi dilakukan secara paralel sehingga cocok digunakan pada sistem modern yang memerlukan kinerja tinggi. Selain itu, AES telah menjadi standar kriptografi yang diakui secara luas dan didukung oleh banyak perangkat keras modern melalui instruksi khusus, sehingga mampu memberikan keamanan dan performa yang optimal.

### **2.1.3. Key Derivation Function (KDF)**

Key Derivation Function (KDF) adalah fungsi kriptografi yang digunakan untuk mengubah *password* atau *passphrase* yang dimasukkan pengguna menjadi kunci enkripsi yang lebih kuat dan aman. Proses ini diperlukan karena *password* yang dibuat manusia umumnya memiliki tingkat keacakan yang rendah dan lebih mudah ditebak. Dalam pembentukan kunci, KDF memanfaatkan *salt*, yaitu data acak yang digabungkan dengan *password* agar pengguna yang memiliki *password* yang sama tetap menghasilkan kunci

yang berbeda serta untuk mencegah serangan seperti *rainbow table*. Selain itu, KDF juga menerapkan proses perhitungan berulang (*iterasi*) yang sengaja dirancang untuk memperlambat pembentukan kunci sehingga serangan *brute-force* menjadi lebih sulit dan membutuhkan sumber daya yang lebih besar. Secara umum, proses KDF dimulai dengan menggabungkan *master password* dan *salt*, kemudian memprosesnya menggunakan algoritma seperti PBKDF2 atau Argon2 hingga menghasilkan kunci kriptografi yang siap digunakan dalam proses enkripsi dan dekripsi data.

#### **2.1.4. Cryptographically Secure Pseudorandom Number Generator (CSPRNG)**

Dalam pengembangan sistem keamanan, pembangkitan elemen acak seperti kunci enkripsi, token sesi, maupun pembuatan kata sandi otomatis memerlukan generator khusus yang dikenal dengan istilah *Cryptographically Secure Pseudorandom Number Generator* (CSPRNG). Berbeda dengan generator bilangan acak biasa yang umumnya dijumpai dalam fungsi bawaan bahasa pemrograman untuk kebutuhan simulasi, CSPRNG wajib memiliki sifat statistik yang ketat agar angka yang dihasilkan benar-benar acak dan tidak berpola. Terdapat beberapa perbedaan fundamental dan implementasi khusus dari CSPRNG yang membedakannya dengan generator standar:

- Perbedaan dengan PRNG Biasa: Generator bilangan acak biasa (PRNG) mengutamakan kecepatan eksekusi dan keunikan distribusi statistik, namun rentan ditebak apabila pihak luar berhasil mengetahui nilai *seed* awal atau urutan beberapa angka sebelumnya. Sebaliknya, CSPRNG wajib lolos uji ketidakpastian (*next-bit test*) dan memiliki sifat *forward secrecy*, yang berarti walaupun status internal generator saat ini bocor, pihak luar tetap tidak akan bisa memprediksi angka acak yang dihasilkan pada masa lalu maupun masa depan.
- Pembangkitan Password Otomatis: Di dalam sistem pengelola kata sandi, CSPRNG dimanfaatkan secara intensif untuk memproduksi string acak ber-entropi tinggi ketika pengguna menggunakan fitur pembuat kata sandi otomatis (*password generator*). Karakter acak yang disebarkan oleh CSPRNG memastikan kata sandi baru tersebut bersih dari pola bahasa manusia, sehingga menjadikannya sangat tangguh terhadap serangan berbasis kamus (*dictionary attack*).



### 2.1.5. SQLite dan Arsitektur Client-Server

Implementasi aplikasi modern umumnya mengandalkan arsitektur *client-server* sebagai fondasi distribusi beban komputasi dan datanya. Dalam model ini, sistem dibagi menjadi dua entitas utama, yaitu *client* yang bertindak sebagai antarmuka pengguna dan peminta layanan (seperti aplikasi di smartphone atau komputer), serta *server* yang bertindak sebagai penyedia layanan terpusat yang berjalan di latar belakang internet untuk mengelola penyimpanan, validasi, dan sinkronisasi data antarperangkat. Dalam konteks manajemen data, pembagian tugas dan mekanisme penyimpanan pada arsitektur ini diatur melalui ketentuan berikut:

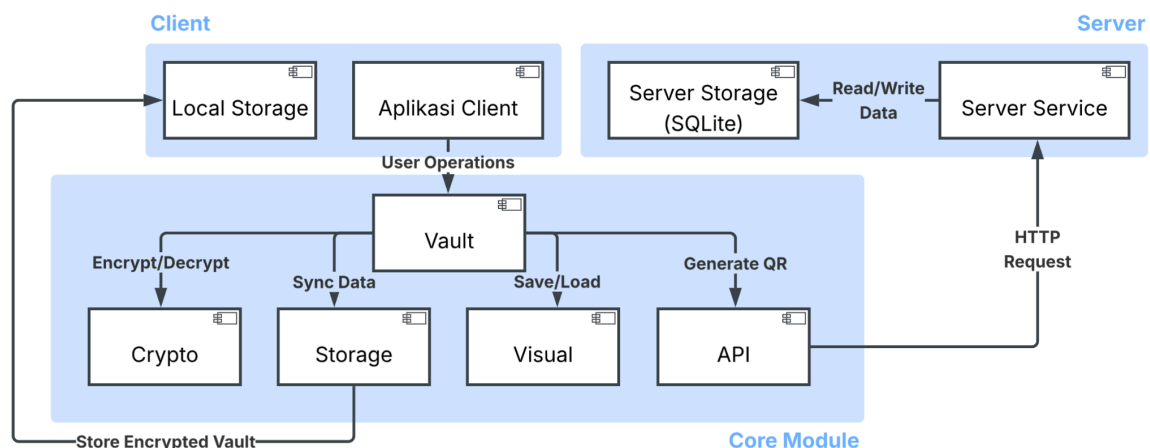
- Pembagian Tugas Client dan Server: Komponen *client* bertanggung jawab penuh dalam menampilkan Antarmuka Pengguna (GUI), menerima input master password, serta melakukan proses enkripsi dan dekripsi data secara lokal sebelum dikirimkan. Di sisi lain, *server* bertugas menerima kiriman paket data terenkripsi tersebut, mengelola autentikasi sesi login akun, serta memfasilitasi proses sinkronisasi data agar dapat diakses dari berbagai perangkat *client* yang berbeda.
- Penyimpanan Data menggunakan SQLite: Untuk manajemen penyimpanan data lokal pada sisi *client*, sistem menggunakan SQLite, sebuah sistem manajemen basis data relasional (RDBMS) tertanam (*embedded*). Berbeda dengan database tradisional seperti MySQL yang memerlukan proses server mandiri, SQLite menyimpan seluruh struktur tabel dan datanya ke dalam satu file tunggal di memori internal perangkat, menjadikannya sangat efisien untuk menyimpan *cache* brankas sandi terenkripsi.

Hubungan kerja antara *client* dan *server* ini umumnya diperkuat dengan menerapkan konsep *Zero-Knowledge Server*. Melalui arsitektur ini, server dirancang secara spesifik agar tidak memiliki kemampuan atau kunci enkripsi apa pun untuk membaca data sensitif yang dititipkan kepadanya. Karena seluruh proses enkripsi dilakukan di sisi *client* sebelum data dikirimkan lewat jaringan, server hanya melihat dan menyimpan data dalam bentuk teks acak (*ciphertext*). Hal ini memastikan bahwa apabila data di dalam server bocor atau disita oleh pihak ketiga, informasi sensitif pengguna akan tetap aman karena kunci dekripsinya hanya dimiliki oleh *client*.

## 2.2. Perancangan Sistem

### 2.2.1. Arsitektur dan Penyimpanan Data

Sistem tugas ini dirancang menggunakan arsitektur client-server terdistribusi yang mengutamakan keamanan dan kerahasiaan data pengguna. Untuk menerapkan prinsip *zero-knowledge*, seluruh proses kriptografi yang melibatkan informasi sensitif dilakukan pada sisi klien sehingga server tidak pernah menerima, memproses, maupun menyimpan *master password* dan *master key* pengguna. Secara umum, arsitektur sistem terdiri atas tiga lapisan utama, yaitu Client, Core Module, dan Server. Lapisan Client menyediakan antarmuka yang digunakan pengguna untuk mengakses dan mengelola data kredensial. Lapisan Core Module berisi logika utama sistem yang mencakup pengelolaan vault, layanan kriptografi, penyimpanan data, komunikasi jaringan, dan pembangkitan media pemulihan. Sementara itu, lapisan Server menyediakan layanan penyimpanan data terenkripsi yang dapat diakses oleh klien tanpa mengetahui isi data yang disimpan.



Gambar 2.1 Diagram Komponen dan Hubungan Antar-Modul Aplikasi

Berdasarkan Gambar 2.1, Client Application berinteraksi dengan Vault Manager sebagai pusat pengelolaan data kredensial. Vault Manager mengoordinasikan penggunaan beberapa modul pendukung sesuai kebutuhan proses yang dijalankan. Crypto Module digunakan untuk melaksanakan fungsi kriptografi seperti derivasi kunci menggunakan Argon2id, enkripsi dan dekripsi data menggunakan AES-128-GCM, serta pembagian dan rekonstruksi rahasia menggunakan *Shamir's Secret Sharing*. Storage Module bertanggung jawab mengelola penyimpanan data pada Local Storage, sedangkan API Module menangani

komunikasi HTTP antara klien dan server. Selain itu, Visual Module digunakan untuk menghasilkan media pemulihan berbasis QR Code dan *Visual Cryptography* yang merepresentasikan *recovery share* dalam bentuk visual.

Pada sisi server, **Server Service** menerima permintaan yang dikirimkan oleh klien melalui API dan mengelola proses penyimpanan maupun pengambilan data pada **Server Storage**. Seluruh data yang diterima server telah berada dalam kondisi terenkripsi sehingga server hanya berfungsi sebagai penyedia layanan penyimpanan tanpa memiliki kemampuan untuk mengakses isi data pengguna.

Untuk mengurangi risiko *single point of failure*, informasi yang diperlukan untuk membuka vault tidak disimpan pada satu lokasi penyimpanan. Data yang berkaitan dengan proses rekonstruksi kunci dibagi ke beberapa lokasi sehingga tidak ada satu komponen pun yang memiliki seluruh informasi yang diperlukan untuk memperoleh *master key*. Penyimpanan lokal digunakan untuk menyimpan komponen yang dibutuhkan dalam proses autentikasi dan rekonstruksi rahasia pada sisi klien, sedangkan penyimpanan server digunakan untuk menyimpan data cadangan serta vault yang telah terenkripsi. Rincian pembagian data pada setiap lokasi penyimpanan ditunjukkan pada Tabel 2.1.

*Tabel 2.1 Pembagian Data pada Lokasi Penyimpanan Sistem*

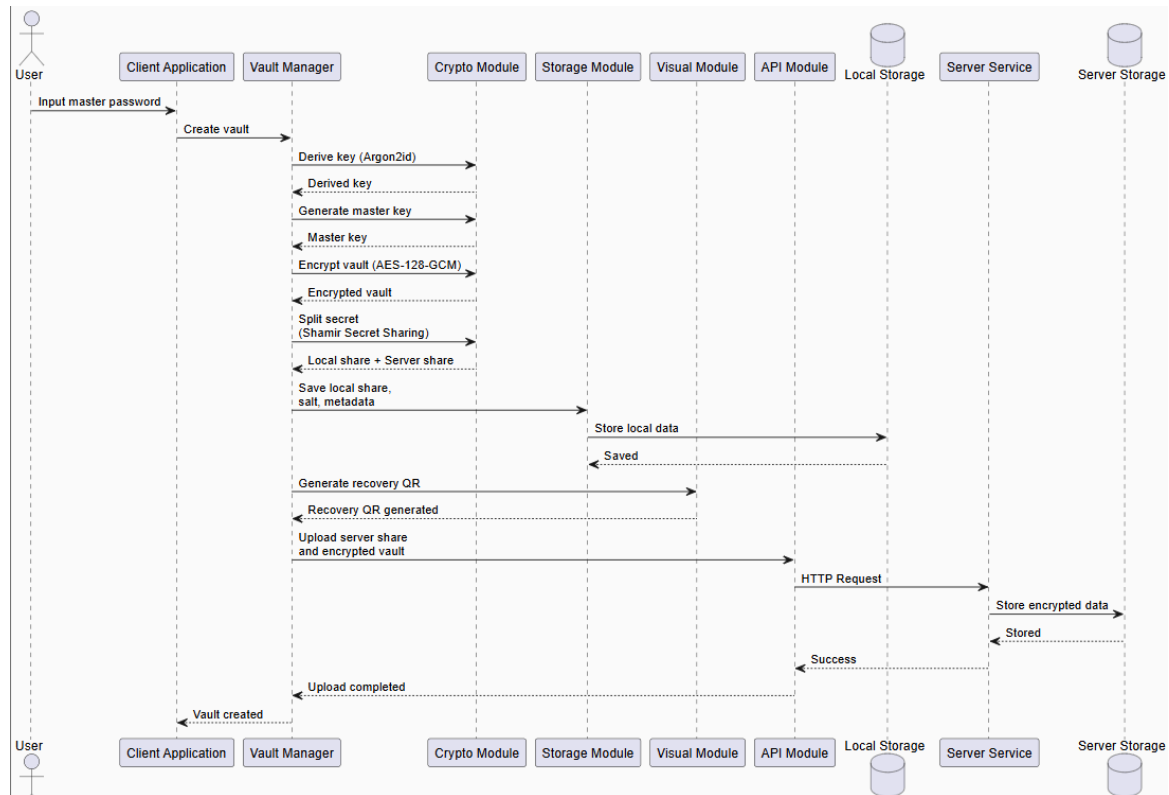
| Lokasi                             | Data yang Disimpan                                                                      | Fungsi                                                                                                                            |
|------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Local Store                        | <i>local share</i> , <i>salt</i> , parameter KDF, dan metadata lokal                    | Menyimpan komponen yang diperlukan untuk proses derivasi kunci dan rekonstruksi rahasia pada sisi klien.                          |
| Server Store                       | <i>server share</i> , vault terenkripsi ( <i>encrypted vault</i> ), dan <i>nonce</i>    | Menyimpan data cadangan dan data vault yang telah dienkripsi tanpa mengetahui isi informasi pengguna.                             |
| Memori Aplikasi ( <i>runtime</i> ) | <i>master password</i> , kunci hasil Argon2id, dan <i>master key</i> hasil rekonstruksi | Digunakan sementara selama proses autentikasi, enkripsi, dekripsi, dan rekonstruksi rahasia serta tidak disimpan secara permanen. |

Berdasarkan Tabel 2.1, tidak terdapat satu lokasi penyimpanan yang memiliki seluruh komponen yang diperlukan untuk membuka vault. Oleh karena itu, kompromi pada salah satu media penyimpanan tidak secara langsung memungkinkan penyerang

memperoleh akses penuh terhadap data pengguna. Sistem menerapkan dua lapisan proteksi kriptografi yang saling melengkapi. Pada lapisan pertama, *master password* diproses menggunakan Argon2id untuk menghasilkan kunci derivasi yang digunakan dalam perlindungan data yang tersimpan pada sisi klien. Pada lapisan kedua, *master key* yang berhasil direkonstruksi melalui mekanisme *Shamir's Secret Sharing* digunakan untuk mengenkripsi dan mendekripsi data vault menggunakan AES-128-GCM. Dengan pendekatan ini, komponen autentikasi dan data vault terlindungi secara terpisah sehingga keamanan sistem tetap terjaga meskipun salah satu lokasi penyimpanan mengalami kebocoran data.

### **2.2.2. Perancangan Alur Operasional Sistem**

Keandalan sistem pengelola kata sandi pada tugas ini tidak hanya ditentukan oleh kekuatan algoritma kriptografi yang digunakan, tetapi juga oleh perancangan alur operasional yang mengatur bagaimana data diproses, disimpan, dan dipulihkan. Sistem dirancang agar *master key* tidak pernah disimpan secara utuh pada media penyimpanan mana pun. Kunci tersebut hanya berada sementara di memori aplikasi selama proses autentikasi dan akses vault berlangsung. Ketika pengguna ingin mengakses data, *master key* direkonstruksi kembali menggunakan mekanisme *Shamir's Secret Sharing* setelah pengguna berhasil menyediakan *master password* yang benar dan seluruh komponen rahasia yang diperlukan tersedia. Dengan pendekatan ini, sistem menerapkan prinsip *zero-knowledge*, yaitu server tidak pernah mengetahui maupun menyimpan informasi yang cukup untuk membuka data pengguna. Berdasarkan kebutuhan sistem, alur operasional dibagi menjadi tiga skenario utama, yaitu pembuatan vault, akses normal terhadap vault, dan penanganan kegagalan rekonstruksi rahasia.



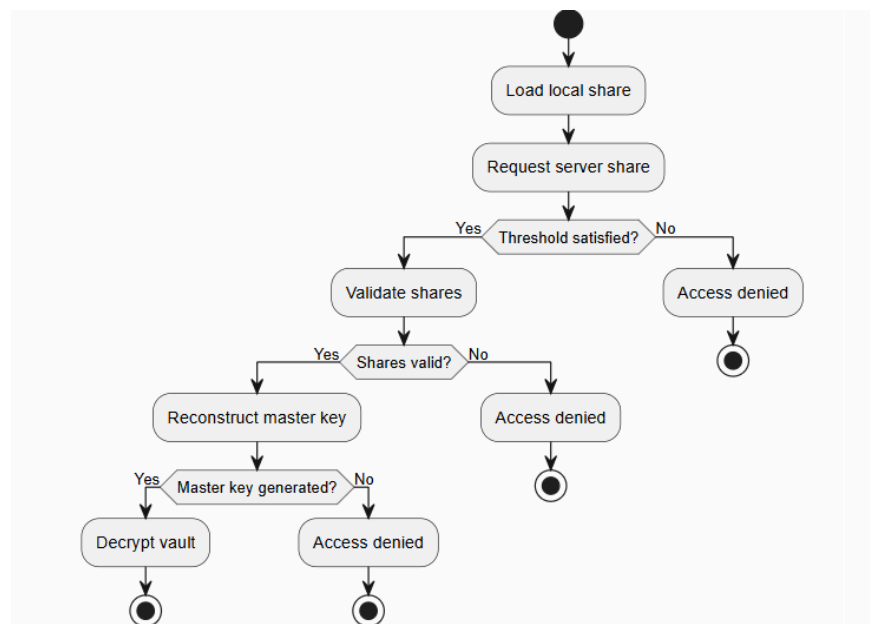
Gambar 2.2 Sequence Diagram Perancangan Alur Pembuatan Vault

Alur pertama menggambarkan proses pembuatan vault yang dilakukan saat pengguna pertama kali menginisialisasi sistem. Seperti ditunjukkan pada Gambar 2.2, proses dimulai ketika pengguna memasukkan *master password* melalui *Client Application*. Selanjutnya, *Vault Manager* mengoordinasikan proses pembentukan vault dengan memanfaatkan *Crypto Module*. Modul tersebut melakukan derivasi kunci menggunakan Argon2id, menghasilkan *master key*, mengenkripsi data vault menggunakan AES-128-GCM, serta membagi *master key* menjadi beberapa *share* melalui mekanisme *Shamir's Secret Sharing*. Setelah proses pembagian rahasia selesai, *Storage Module* menyimpan *local share*, *salt*, dan metadata pendukung ke *Local Storage*. Pada saat yang sama, *Visual Module* menghasilkan media pemulihan berupa QR Code berbasis *Visual Cryptography* yang dapat digunakan sebagai sarana pemulihan apabila diperlukan. Selanjutnya, *API Module* mengirimkan *server share* dan data vault yang telah terenkripsi ke *Server Service* untuk disimpan pada *Server Storage*. Setelah seluruh proses berhasil diselesaikan, vault siap digunakan untuk menyimpan dan mengelola data kredensial pengguna.



Gambar 2.3 Flow Diagram Akses Normal Sistem

Alur kedua menggambarkan proses akses normal ketika pengguna ingin melihat, menambah, mengubah, atau menghapus data kredensial yang tersimpan pada vault. Seperti ditunjukkan pada Gambar 2.3, proses diawali dengan pemuatan metadata lokal yang diperlukan untuk proses derivasi kunci. Pengguna kemudian memasukkan *master password* yang diproses menggunakan Argon2id untuk menghasilkan kunci derivasi. Kunci tersebut digunakan untuk memperoleh kembali *local share* yang tersimpan pada perangkat pengguna. Setelah *local share* berhasil diperoleh, aplikasi meminta *server share* dan data vault terenkripsi dari server. Kedua *share* tersebut kemudian direkonstruksi menggunakan mekanisme *Shamir's Secret Sharing* untuk membentuk kembali *master key*. Apabila rekonstruksi berhasil, *master key* digunakan untuk mendekripsi vault menggunakan AES-128-GCM sehingga data kredensial dapat ditampilkan dan dikelola oleh pengguna. Seluruh proses kriptografi dilakukan pada sisi klien sehingga server tidak pernah memperoleh akses terhadap data dalam bentuk *plaintext*.



Gambar 2.4 Flow Diagram Penanganan Kegagalan Rekonstruksi Rahasia

Alur ketiga menggambarkan mekanisme pengendalian keamanan ketika proses rekonstruksi rahasia tidak dapat dilakukan. Seperti ditunjukkan pada Gambar 2.4, sistem

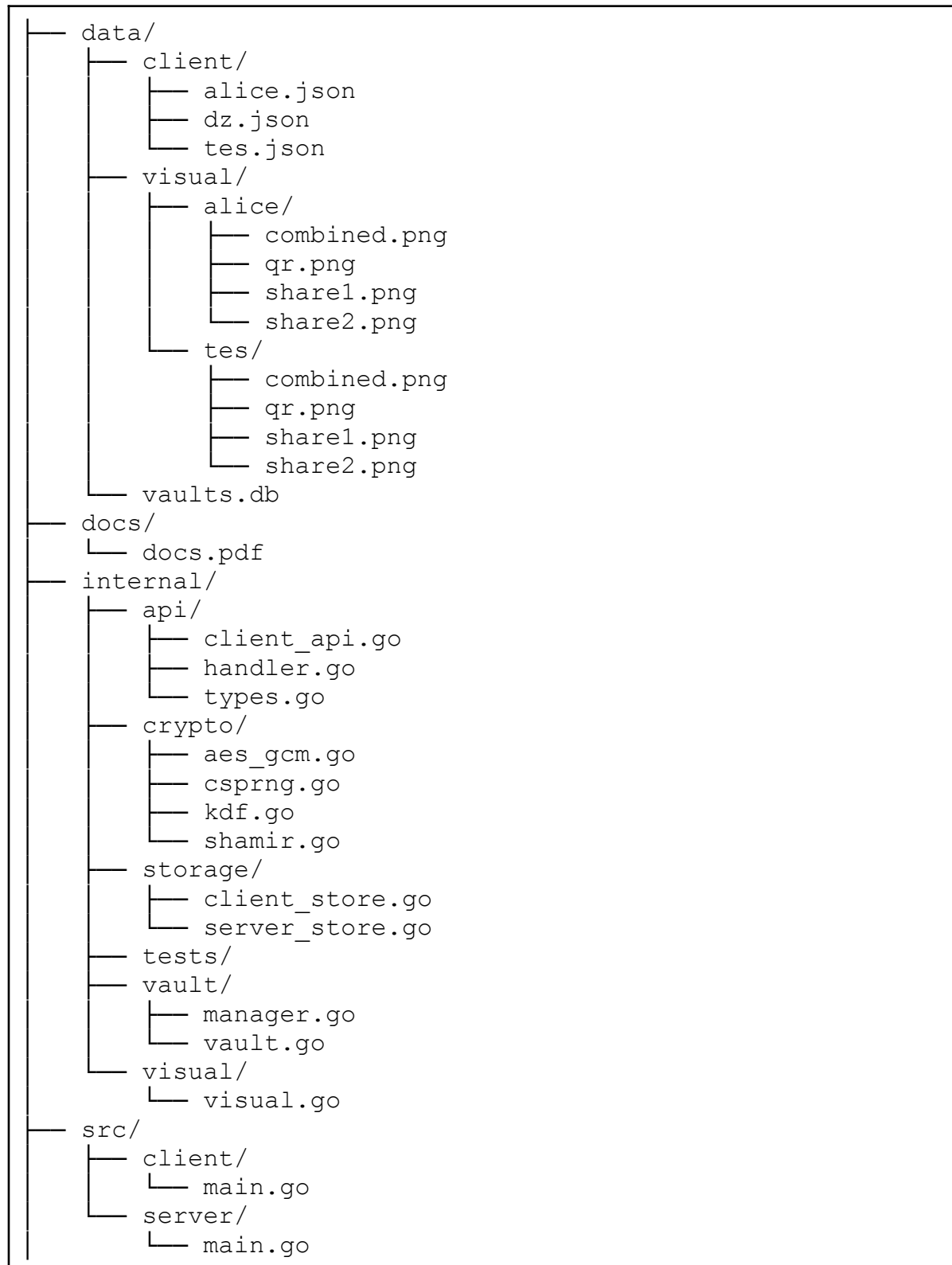
terlebih dahulu memuat *local share* dan meminta *server share* yang diperlukan untuk proses rekonstruksi. Sistem kemudian memeriksa ketersediaan *share* yang dibutuhkan sesuai nilai *threshold* yang telah ditentukan. Apabila jumlah *share* tidak mencukupi, proses rekonstruksi tidak dapat dilakukan dan akses akan langsung ditolak. Jika *share* tersedia, sistem melanjutkan proses rekonstruksi *master key*. Kegagalan pada tahap ini, baik akibat kerusakan data maupun ketidaksesuaian nilai *share*, menyebabkan *master key* tidak dapat dibentuk kembali sehingga proses dekripsi vault tidak dapat dilanjutkan. Akibatnya, sistem akan menolak akses terhadap data pengguna. Mekanisme tersebut memastikan bahwa data sensitif tetap terlindungi meskipun sebagian *share* hilang, rusak, atau berhasil diperoleh oleh pihak yang tidak berwenang.

Ketiga alur tersebut menunjukkan bahwa keamanan sistem tidak hanya bergantung pada algoritma kriptografi yang digunakan, tetapi juga pada pemisahan komponen rahasia ke beberapa lokasi penyimpanan dan mekanisme rekonstruksi yang terkontrol. Dengan mengombinasikan Argon2id, AES-128-GCM, dan *Shamir's Secret Sharing*, sistem mampu menjaga kerahasiaan data pengguna sekaligus mempertahankan kemampuan pemulihan akses tanpa mengorbankan prinsip *zero-knowledge* yang menjadi dasar perancangan arsitektur sistem.

### **2.2.3. Perancangan Modul Sistem**

Untuk membangun aplikasi *zero-knowledge password manager* yang aman, terstruktur, dan mudah dipelihara, kode program disusun berdasarkan prinsip *Separation of Concerns* (SoC). Prinsip ini membagi sistem ke dalam beberapa modul yang memiliki tanggung jawab spesifik sehingga setiap komponen dapat dikembangkan dan dikelola secara independen. Melalui pendekatan tersebut, fungsi kriptografi, penyimpanan data, komunikasi jaringan, pengelolaan vault, serta layanan visualisasi dipisahkan ke dalam modul yang berbeda. Selain meningkatkan keterbacaan dan pemeliharaan kode, pemisahan modul juga membantu mengurangi ketergantungan antar-komponen (*coupling*) serta meningkatkan keamanan karena fungsi kriptografi dapat diisolasi dari komponen lain yang tidak memerlukan akses langsung terhadap proses pengelolaan kunci. Berdasarkan prinsip tersebut, struktur direktori proyek dirancang untuk merepresentasikan pembagian tanggung jawab setiap modul. Struktur ini memisahkan komponen aplikasi klien, layanan server,

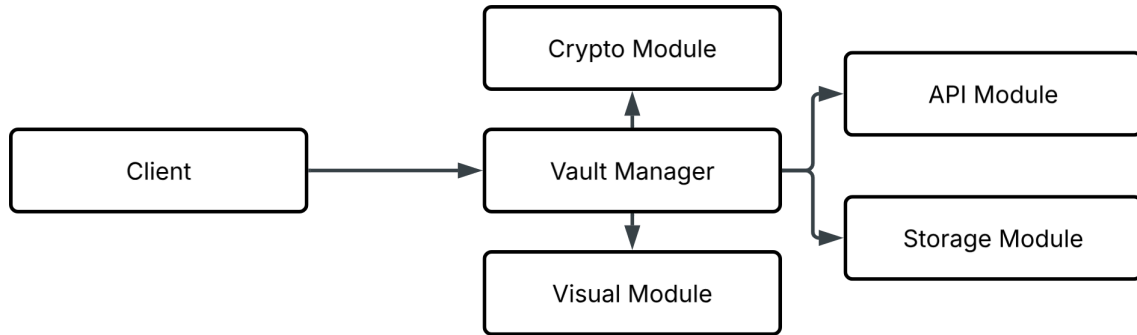
logika inti sistem, penyimpanan data, dan aset pendukung ke dalam direktori yang berbeda sebagaimana ditunjukkan pada Struktur Direktori Proyek berikut.





|     |           |
|-----|-----------|
| ├── | go.mod    |
| ├── | go.sum    |
| └── | README.md |

Untuk memperjelas hubungan antar-modul dan aliran interaksi di dalam sistem, rancangan arsitektur modul ditunjukkan pada Gambar 2.5.



*Gambar 2.5 Arcitecture Diagram Modul Sistem*

Berdasarkan struktur direktori dan diagram arsitektur pada Gambar 2.5, proyek dibagi menjadi dua bagian utama, yaitu direktori `src/` dan `internal/`. Direktori `src/` berisi *entry point* aplikasi yang digunakan untuk menjalankan layanan pada sisi klien maupun server. Sementara itu, direktori `internal/` menyimpan seluruh logika inti sistem yang diimplementasikan melalui beberapa modul, seperti *Vault Manager*, *Crypto Module*, *Storage Module*, *API Module*, dan *Visual Module*. Pengorganisasian ini memastikan bahwa setiap modul memiliki tanggung jawab yang jelas dan dapat digunakan kembali tanpa menimbulkan ketergantungan yang berlebihan antar-komponen. Untuk memberikan gambaran yang lebih rinci mengenai fungsi masing-masing modul, Tabel 2.2 menyajikan ringkasan tanggung jawab komponen yang digunakan dalam sistem.

*Tabel 2.2 Pemetaan Fungsi dan Tanggung Jawab Modul Sistem*

| Komponen                   | Modul  | Fungsi dan Tanggung Jawab                                                                                                     |
|----------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------|
| src/client/main.go         | Client | Menjalankan aplikasi klien, menyediakan antarmuka <i>command-line interface (CLI)</i> , serta menerima masukan dari pengguna. |
| src/server/main.go         | Server | Menjalankan layanan server dan menangani permintaan API yang dikirim oleh klien.                                              |
| internal/api/client_api.go | API    | Mengelola komunikasi HTTP dari klien ke                                                                                       |

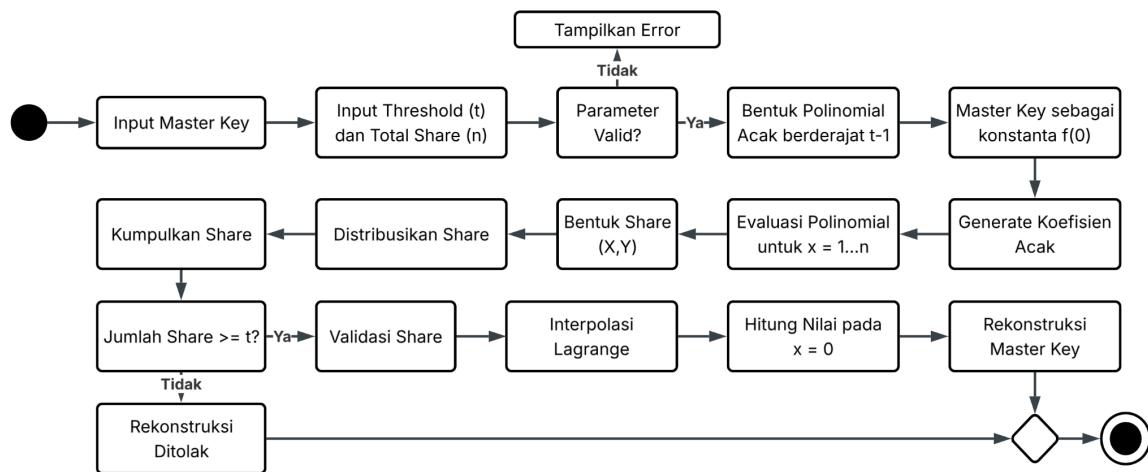
|                                  |                |                                                                                                                                        |
|----------------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------|
|                                  | Layer          | server melalui pengiriman permintaan API.                                                                                              |
| internal/api/handler.go          | API Layer      | Menangani <i>routing</i> dan pemrosesan permintaan API serta mengirimkan respons HTTP kepada klien.                                    |
| internal/api/types.go            | API Layer      | Mendefinisikan struktur data yang digunakan dalam pertukaran data API, seperti <i>StoreRequest</i> dan <i>VaultResponse</i> .          |
| internal/vault/manager.go        | Vault Module   | Mengelola logika utama brankas, termasuk proses enkripsi, dekripsi, dan pemulihan melalui <i>backup mode</i> .                         |
| internal/vault/vault.go          | Vault Module   | Mendefinisikan struktur data brankas, seperti <i>Vault</i> dan <i>Entry</i> , dalam format JSON.                                       |
| internal/crypto/aes_gcm.go       | Crypto Module  | Mengimplementasikan enkripsi dan dekripsi data brankas menggunakan algoritma AES-128-GCM.                                              |
| internal/crypto/csprng.go        | Crypto Module  | Menghasilkan nilai acak dan kata sandi yang aman menggunakan <i>Cryptographically Secure Pseudorandom Number Generator</i> (CSPRNG).   |
| internal/crypto/kdf.go           | Crypto Module  | Menurunkan kunci enkripsi dari <i>master password</i> menggunakan fungsi derivasi kunci Argon2id.                                      |
| internal/crypto/shamir.go        | Crypto Module  | Mengimplementasikan algoritma <i>Shamir's Secret Sharing</i> untuk membagi dan merekonstruksi <i>master key</i> .                      |
| internal/storage/client_store.go | Storage Module | Mengelola penyimpanan lokal terenkripsi di sisi klien, termasuk <i>share</i> , <i>salt</i> , <i>nonce</i> , dan <i>backup vault</i> .  |
| internal/storage/server_store.go | Storage Module | Mengelola penyimpanan data di sisi server menggunakan SQLite untuk menyimpan <i>server share</i> dan data brankas terenkripsi.         |
| internal/visual/visual.go        | Visual Module  | Mengonversi <i>recovery share</i> menjadi QR Code dan membaginya menjadi dua citra menggunakan skema <i>Visual Cryptography</i> (2,2). |
| internal/tests/                  | Testing        | Menyediakan kumpulan pengujian otomatis                                                                                                |

|  |  |                                                                            |
|--|--|----------------------------------------------------------------------------|
|  |  | (unit testing) untuk memverifikasi keakuratan dan keandalan fungsi sistem. |
|--|--|----------------------------------------------------------------------------|

## 2.3. Implementasi Sistem

### 2.3.1. Implementasi Shamir Secret Sharing

Sebelum menjelaskan implementasi pada berkas `shamir.go`, alur proses pembagian dan rekonstruksi rahasia dapat dilihat pada Gambar 2.6. Diagram tersebut menunjukkan bagaimana *master key* dipecah menjadi beberapa *share* menggunakan polinomial acak, kemudian direkonstruksi kembali ketika jumlah *share* yang terkumpul telah memenuhi nilai *threshold*.



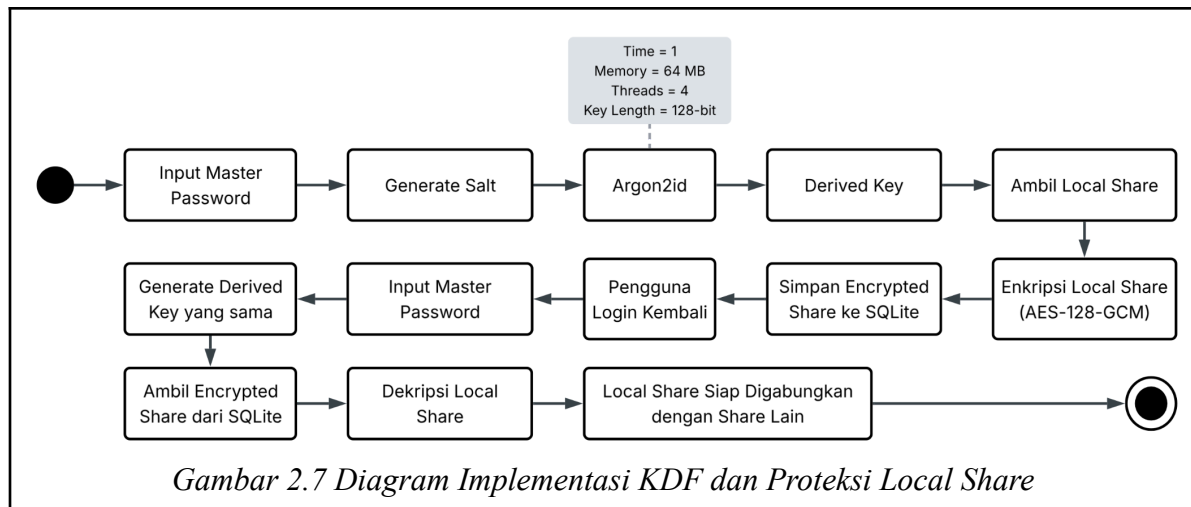
Gambar 2.6 Diagram Implementasi Shamir Secret Sharing

Berdasarkan alur pada Gambar 2.6, implementasi skema pembagian data rahasia pada sistem tugas ini diwujudkan melalui berkas kode sumber `shamir.go`. Modul ini bertanggung jawab mengelola seluruh proses pembentukan dan rekonstruksi *share* menggunakan operasi matematika pada *Galois Field*  $GF(256)$ . Pada tahap pembagian rahasia, fungsi `SplitSecret()` menerima *master key*, nilai *threshold* ( $t$ ), dan jumlah *share* ( $n$ ), kemudian melakukan validasi parameter sebelum membentuk polinomial acak berderajat  $t-1$ . Nilai rahasia ditempatkan sebagai konstanta polinomial, sedangkan koefisien lainnya dibangkitkan secara acak menggunakan pustaka kriptografi bawaan Go. Selanjutnya, fungsi menghitung nilai polinomial pada beberapa titik untuk menghasilkan objek *Share* yang berisi pasangan koordinat  $(X,Y)$ . Pada tahap rekonstruksi, fungsi `CombineShares()` menerima sekumpulan *share* yang telah memenuhi batas minimal *threshold*. Fungsi ini

memanfaatkan metode interpolasi Lagrange melalui fungsi internal `lagrangeInterpolation()` untuk menghitung kembali nilai polinomial pada titik  $x = 0$  sehingga diperoleh *master key* asli. Selama proses rekonstruksi, sistem juga melakukan validasi terhadap setiap *share* yang digunakan. Apabila ditemukan koordinat yang duplikat atau data *share* yang telah dimodifikasi, proses rekonstruksi akan gagal dan sistem akan mengembalikan pesan kesalahan.

### 2.3.2. Implementasi KDF dan Proteksi Local Share

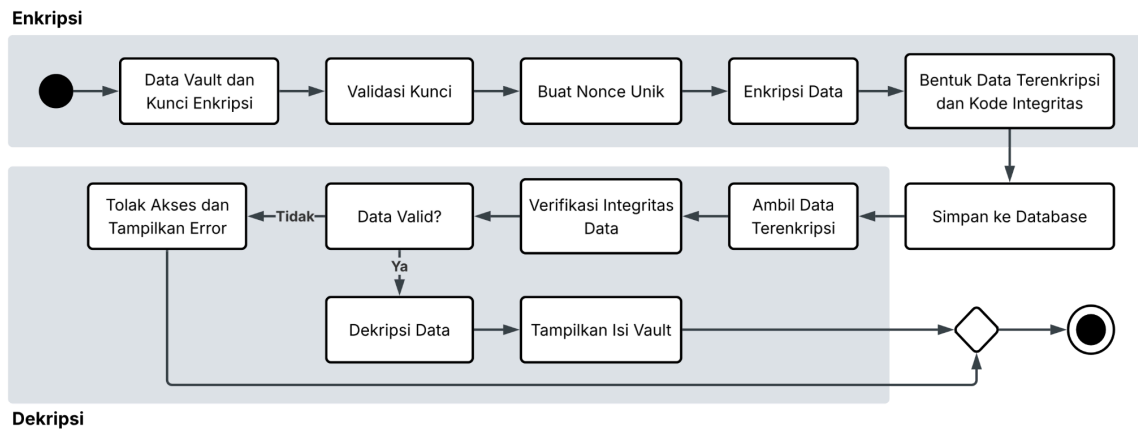
Untuk memperjelas mekanisme pembentukan kunci enkripsi dan perlindungan *local share*, Gambar 2.7 menunjukkan alur proses yang dijalankan oleh modul `kdf.go`. Diagram tersebut menggambarkan bagaimana *master password* diubah menjadi kunci kriptografi menggunakan Argon2id, kemudian digunakan untuk mengenkripsi *local share* sebelum disimpan ke basis data lokal. Algoritma Argon2id dipilih sebagai *Key Derivation Function* (KDF) karena menggabungkan keunggulan dua variannya, yaitu Argon2i yang lebih tahan terhadap *side-channel attack* dan Argon2d yang lebih tahan terhadap serangan menggunakan perangkat keras paralel seperti GPU dan ASIC. Kombinasi tersebut membuat Argon2id mampu memberikan tingkat keamanan yang tinggi sekaligus meningkatkan ketahanan terhadap serangan *brute-force*. Selain itu, Argon2id merupakan algoritma yang bersifat *memory-hard*, yaitu membutuhkan penggunaan memori yang cukup besar selama proses derivasi kunci. Karakteristik ini membuat upaya penyerangan menjadi lebih mahal dan sulit dilakukan dalam skala besar. Argon2id juga menyediakan parameter yang dapat dikonfigurasi, seperti waktu komputasi (*time cost*), penggunaan memori (*memory cost*), dan jumlah *thread* (*parallelism*), sehingga tingkat keamanan dapat disesuaikan dengan kebutuhan dan kemampuan perangkat yang digunakan.



Berdasarkan alur pada Gambar 2.7, mekanisme penurunan kunci dan perlindungan *local share* diimplementasikan pada berkas *kdf.go*. Fungsi utama yang digunakan adalah *DeriveKey()*, yang memanfaatkan algoritma *Argon2id* untuk menghasilkan kunci kriptografi dari *master password* pengguna. Sebelum proses derivasi dilakukan, sistem membangkitkan nilai *salt* unik menggunakan fungsi *GenerateSalt()*. Selanjutnya, *master password* dan *salt* diproses menggunakan parameter yang telah ditentukan melalui *DefaultKDFParams()*, yaitu waktu komputasi, penggunaan memori, jumlah *thread*, dan panjang kunci keluaran sebesar 128 bit. Kunci hasil derivasi tidak pernah disimpan secara permanen di dalam sistem. Sebaliknya, kunci tersebut hanya digunakan sementara di memori untuk mengenkripsi *local share* menggunakan *AES-128-GCM* sebelum disimpan ke basis data *SQLite* lokal. Ketika proses rekonstruksi rahasia dilakukan, pengguna harus memasukkan kembali *master password* yang sama agar sistem dapat menghasilkan *derived key* yang identik untuk mendekripsi *local share*. Dengan mekanisme ini, meskipun berkas basis data lokal berhasil diakses oleh pihak yang tidak berwenang, isi *local share* tetap tidak dapat dibaca tanpa mengetahui *master password* yang benar.

### 2.3.3. Implementasi AES-128-GCM pada Vault

Untuk memperjelas mekanisme perlindungan data pada *vault*, Gambar 2.8 menunjukkan alur proses enkripsi dan dekripsi data yang diterapkan pada sistem. Proses ini memastikan bahwa data kata sandi disimpan dalam bentuk terenkripsi dan hanya dapat dibaca kembali oleh pengguna yang memiliki kunci yang valid.

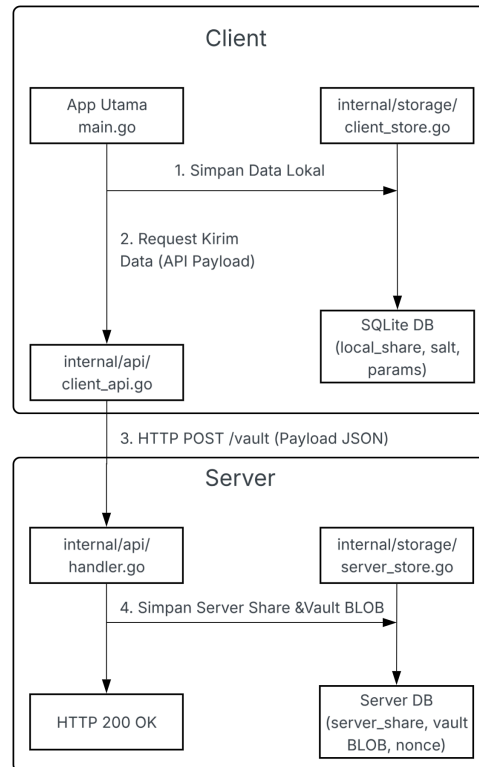


Gambar 2.8 Diagram Penyimpanan Data dan Server

Berdasarkan alur pada Gambar 2.8, proses perlindungan data *vault* diimplementasikan pada berkas `aes_gcm.go` melalui fungsi `EncryptAESGCM()` dan `DecryptAESGCM()`. Fungsi `EncryptAESGCM()` menerima masukan berupa kunci hasil rekonstruksi *secret sharing* dan data *plaintext* yang akan diamankan. Fungsi ini melakukan validasi panjang kunci, membangun objek AES menggunakan `aes.NewCipher()`, kemudian menginisialisasi mode GCM melalui `cipher.NewGCM()`. Selanjutnya sistem membangkitkan *nonce* unik menggunakan `rand.Read()` dan melakukan proses enkripsi menggunakan `gcm.Seal()`. Hasil proses ini berupa *ciphertext* yang telah dilengkapi dengan *authentication tag* sehingga kerahasiaan dan integritas data dapat dijaga secara bersamaan. Pada proses pembacaan data, fungsi `DecryptAESGCM()` menerima kunci, *nonce*, dan *ciphertext* yang tersimpan pada basis data. Setelah objek AES-GCM dibentuk kembali, fungsi menjalankan `gcm.Open()` untuk memverifikasi *authentication tag* sekaligus melakukan dekripsi. Jika data telah dimodifikasi atau kunci yang digunakan tidak sesuai, proses verifikasi akan gagal dan sistem mengembalikan pesan kesalahan tanpa menampilkan isi data yang tersimpan.

#### 2.3.4. Implementasi Penyimpanan Data dan Server

Untuk memperjelas mekanisme penyimpanan data dan komunikasi antara klien dan server, Gambar 2.9 menunjukkan alur proses yang digunakan sistem ketika menyimpan maupun mengambil data *vault*. Diagram tersebut menggambarkan bagaimana data yang telah terenkripsi dikirimkan ke server untuk disimpan pada basis data SQLite serta bagaimana data tersebut diambil kembali saat proses pembukaan *vault* dilakukan.



*Gambar 2.9 Diagram Implementasi Penyimpanan Data dan Sinkronisasi Server*

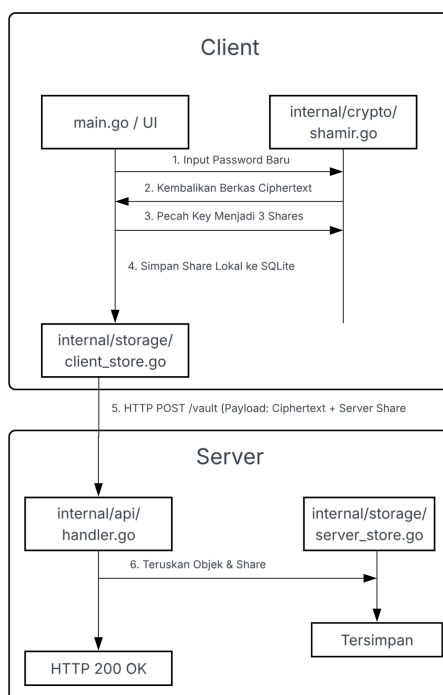
Berdasarkan alur pada Gambar 2.9, implementasi manajemen database dan mekanisme komunikasi *client-server* pada sistem tugas ini diwujudkan melalui kolaborasi berkas kode sumber di dalam paket `internal/storage/` dan `internal/api/`. Pada sisi klien, berkas `client_store.go` bertanggung jawab penuh terhadap pembuatan skema dan pengelolaan basis data relasional lokal tersemat menggunakan SQLite. Fungsi utama di dalam modul ini akan melakukan inisialisasi tabel lokal secara otomatis saat aplikasi pertama kali dijalankan, kemudian menyimpan *local share* terenkripsi, nilai *salt*, serta metadata parameter KDF (Argon2id) ke dalam penyimpanan memori internal perangkat klien secara persisten. Pemilihan SQLite memastikan proses baca-tulis data lokal berjalan sangat cepat, efisien, dan terisolasi tanpa memerlukan dependensi server database eksternal di sisi *endpoint* pengguna.

Sementara itu, untuk memfasilitasi distribusi data menuju arsitektur jaringan luar, berkas `client_api.go` pada lapisan API bertindak sebagai jembatan komunikasi dengan membungkus data *server share*, *ciphertext vault* dalam bentuk objek BLOB, dan nilai *nonce* ke dalam format JSON (*JavaScript Object Notation*). Protokol HTTP dengan metode POST

dikirimkan menuju *endpoint* server secara aman. Pada sisi penerima, layanan server yang ditenagai oleh berkas `handler.go` menerima paket *request* tersebut, melakukan pembedahan *payload* (*parsing*), serta memvalidasi token sesi pengguna. Jika data dinyatakan valid, `handler.go` akan meneruskan objek data tersebut kepada modul `server_store.go` untuk disimpan ke dalam *repository Server Store*. Implementasi ini mempertahankan prinsip ketat *Zero-Knowledge Server*, di mana server hanya berkewajiban mengamankan struktur teks acak (*ciphertext*) tanpa memiliki kapabilitas atau instrumen kriptografi apa pun untuk mengintip isi data kredensial asli milik pengguna.

### 2.3.5. Implementasi Pengelolaan Password

Implementasi modul pengelolaan password merupakan inti dari fungsi aplikasi *password manager*, yang mencakup logika penambahan, enkripsi, dan pembagian rahasia (*secret sharing*). Alur data dan interaksi antarkomponen kode saat pengguna menyimpan sebuah kredensial baru dapat dilihat pada Gambar 2.10.



Gambar 2.10 Diagram Alur Implementasi Pengelolaan dan Pengamanan Password

Berdasarkan arsitektur yang tertera pada Gambar 2.10, fungsi pengelolaan password direalisasikan melalui integrasi erat antara paket kriptografi `internal/crypto/` dan modul penyimpanan data. Ketika pengguna memasukkan data kredensial baru berupa *username*

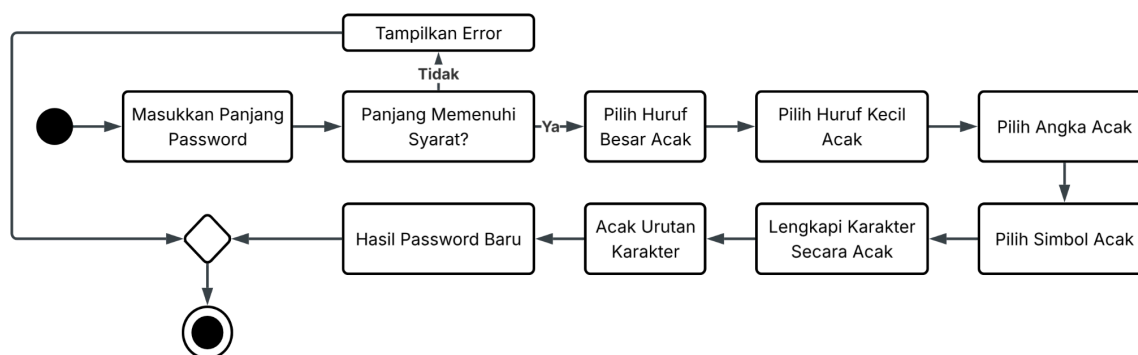


dan *password*, sistem tidak langsung menyimpannya dalam bentuk teks polos (*plaintext*). Berkas *shamir.go* yang berada di dalam paket kriptografi terlebih dahulu melakukan enkripsi simetris menggunakan algoritma AES-GCM (*Advanced Encryption Standard - Galois/Counter Mode*) dengan memanfaatkan kunci turunan dari *master password* pengguna. Hasil dari proses ini berupa sebuah *ciphertext* (vault) yang aman dari pembacaan tidak sah.

Setelah *ciphertext* terbentuk, kunci enkripsi tersebut didekonstruksi menggunakan skema *Shamir's Secret Sharing* dengan parameter ambang batas  $(k, n) = (2, 3)$ . Algoritma pada *shamir.go* memecah kunci menjadi 3 bagian (*shares*) yang saling independen secara matematis. Langkah berikutnya diatur oleh modul utama, di mana berkas *client\_store.go* diperintahkan untuk menyimpan satu buah *share* pertama (*client share*) secara lokal ke dalam database SQLite klien. Dua *share* sisanya (*server shares*), bersama dengan teks *ciphertext* yang terenkripsi tadi, dibungkus menjadi komponen payload JSON oleh berkas *client\_api.go*. Payload tersebut dikirimkan melalui jaringan internet menggunakan protokol HTTP POST menuju *endpoint* server. Melalui mekanisme pemisahan komponen ini, keamanan data terkelola secara desentralisasi, sehingga penyerang tidak akan dapat memulihkan password asli tanpa berhasil mengompromikan sisi klien dan server sekaligus.

### 2.3.6. Implementasi Password Generator dan Mode Backup

Untuk memperjelas mekanisme pembangkitan kata sandi otomatis, Gambar 2.11 menunjukkan alur proses yang digunakan sistem untuk menghasilkan kata sandi yang kuat dan memiliki tingkat keacakan tinggi. Proses ini memastikan bahwa kata sandi yang dihasilkan mengandung kombinasi huruf, angka, dan simbol serta tidak memiliki pola yang mudah ditebak.



*Gambar 2.11 Password Generator dan Mode Backup*

Berdasarkan alur pada Gambar 2.11, proses pembangkitan kata sandi diimplementasikan pada berkas `csprng.go` melalui fungsi utama `GeneratePassword()`. Fungsi ini menerima parameter panjang kata sandi dan melakukan validasi terhadap nilai minimum yang ditetapkan oleh konstanta `minPasswordLength`. Untuk memenuhi kebijakan kompleksitas, fungsi mengambil minimal satu karakter dari kelompok huruf besar, huruf kecil, angka, dan simbol. Pemilihan karakter dilakukan menggunakan fungsi pembantu `randomInt()` yang memanfaatkan sumber bilangan acak aman dari pustaka `crypto/rand`. Setelah seluruh karakter yang diperlukan berhasil dibangkitkan, fungsi `GeneratePassword()` menerapkan algoritma Fisher-Yates *shuffle* untuk mengacak posisi karakter sehingga tidak terbentuk pola tertentu berdasarkan urutan pengambilan karakter awal. Selama proses pengacakan, pemilihan indeks pertukaran karakter juga dilakukan menggunakan fungsi `randomInt()`. Melalui kombinasi penggunaan CSPRNG dan algoritma pengacakan tersebut, sistem mampu menghasilkan kata sandi dengan tingkat entropi yang tinggi sehingga lebih tahan terhadap serangan *dictionary attack* maupun *brute-force attack*.

## 2.4. Pengujian dan Analisis Hasil

Pada tahap pengujian ini, akan dilakukan pengujian sistem ini pada user bernama “bob”, dengan master password “abc”, serta recovery share “A8E0xQWcvDjfqhFwfRyh1H0=”.

### 2.4.1. Menyalakan Server

```
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager> go run ./src/server -db data/vaults.db -addr :8080
Server listening on :8080
```

*Gambar 2.12 Output Berhasil Menyalakan Server*

### 2.4.2. Pengujian Pembuatan Vault

```
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager> go run ./src/client create bob
Master password:
Recovery share (store this safely):
A8E0xQWcvDjfqhFwfRyh1H0=
```

*Gambar 2.13 Output Berhasil Membuat Vault*

### 2.4.3. Pengujian Penyimpanan dan Perlindungan Local Share

```
{
  "encrypted_local_share": "WL87rYLh+mCibRpYAdjiNRqIS7pMr1/a5AZZwQpcTWkh",
  "salt": "IVkeK2m9b9NGWBfwkV/7ww==",
  "local_nonce": "Sgr0NyrEg/FMxav/",
  "backup_vault": "HqDC10aeIlcFcQ6XbKpifFEgxY7ZCYmzQp+zjxt2",
  "backup_nonce": "/40IaX4BejedYb4e"
}
```

*Gambar 2.14 Penyimpanan dan Perlindungan Local Share Dengan Enkripsi*

### 2.4.4. Pengujian Akses Normal

```
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager> go run ./src/client open bob
Master password:
Vault entries: (empty)
```

*Gambar 2.15 Output Berhasil Mengakses Vault dengan*

### 2.4.5. Pengujian Penambahan Data Password

```
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager> go run ./src/client add bob
Master password:
Service name: pengujian
Username/email: bob
Password (leave empty to auto-generate): asd
Entry added and vault updated on server and local backup
```

*Gambar 2.16 Output Berhasil Menambahkan Data Password*

### 2.4.6. Pengujian Pengubahan dan Penghapusan Data Password

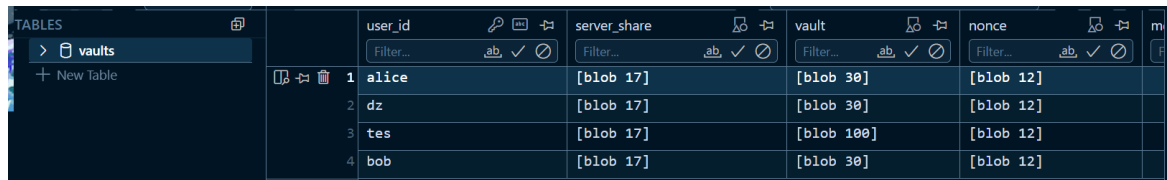
```
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager> go run ./src/client edit bob
Master password:
1) pengujian - bob
2) hapus - bob
Entry number to edit: 1
Service name [pengujian]: pengujian edit
Username/email [bob]: bob
Password (empty = keep, '-' = auto): asdf
Notes []: ini pengujian
Entry updated and vault saved
```

*Gambar 2.17 Output Berhasil Melakukan Perubahan Data Password*

```
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager> go run ./src/client delete bob
Master password:
1) pengujian edit - bob
2) hapus - bob
Entry number to delete: 2
Entry deleted and vault saved
```

*Gambar 2.18 Output Berhasil Melakukan Penghapusan Data Password*

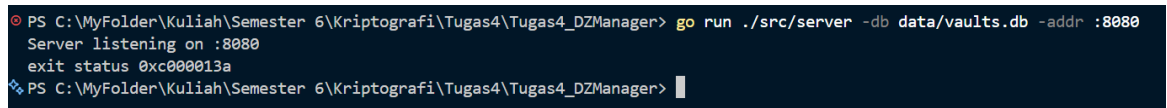
### 2.4.7. Pengujian Penyimpanan Vault pada Server



|   | user_id | server_share | vault      | nonce     |
|---|---------|--------------|------------|-----------|
| 1 | alice   | [blob 17]    | [blob 30]  | [blob 12] |
| 2 | dz      | [blob 17]    | [blob 30]  | [blob 12] |
| 3 | tes     | [blob 17]    | [blob 100] | [blob 12] |
| 4 | bob     | [blob 17]    | [blob 30]  | [blob 12] |

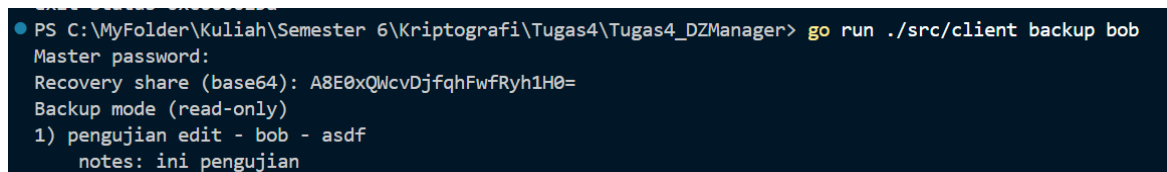
Gambar 2.19 Penyimpanan Vault pada Server (Database SQLite)

### 2.4.8. Pengujian Mode Backup



```
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager> go run ./src/server -db data/vaults.db -addr :8080
Server listening on :8080
exit status 0xc000113a
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager>
```

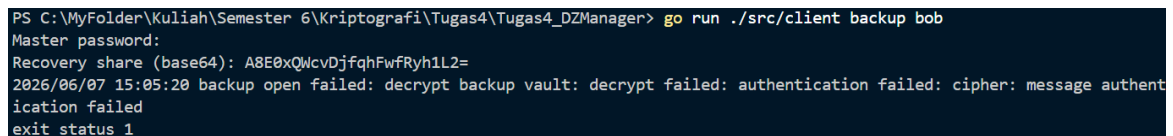
Gambar 2.20 Mematikan Server



```
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager> go run ./src/client backup bob
Master password:
Recovery share (base64): A8E0xQwcvDjfqhFwfRyh1H0=
Backup mode (read-only)
1) pengujian edit - bob - asdf
notes: ini pengujian
```

Gambar 2.21 Output Berhasil Melakukan Recovery dengan Menggunakan Recovery Share

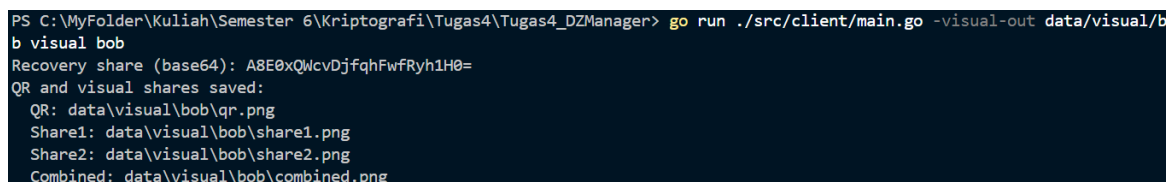
### 2.4.9. Pengujian Kegagalan Pemulihan



```
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager> go run ./src/client backup bob
Master password:
Recovery share (base64): A8E0xQwcvDjfqhFwfRyh1L2=
2026/06/07 15:05:20 backup open failed: decrypt backup vault: decrypt failed: authentication failed: cipher: message authentication failed
exit status 1
```

Gambar 2.21 Output Gagal Melakukan Recovery dengan menggunakan Recovery Share yang Salah

### 2.4.10. Pengujian Bonus Kriptografi Visual (Opsional)

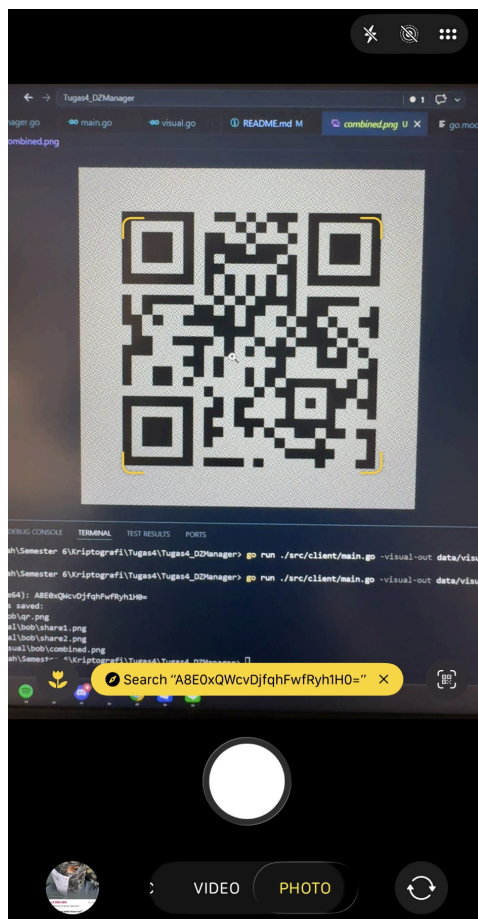


```
PS C:\MyFolder\Kuliah\Semester 6\Kriptografi\Tugas4\Tugas4_DZManager> go run ./src/client/main.go -visual-out data/visual/bob visual bob
Recovery share (base64): A8E0xQwcvDjfqhFwfRyh1H0=
QR and visual shares saved:
QR: data\visual\bob\qr.png
Share1: data\visual\bob\share1.png
Share2: data\visual\bob\share2.png
Combined: data\visual\bob\combined.png
```

Gambar 2.22 Output Berhasil Membuat QR



*Gambar 2.23 Tampilan QR*



*Gambar 2.24 Hasil Scan QR Menampilkan Recovery Share*

## BAB III

### PENUTUP

#### 3.1. Kesimpulan

Berdasarkan seluruh tahapan perancangan, implementasi, dan pengujian yang telah dilakukan pada sistem aplikasi *Password Manager* terdistribusi ini, dapat ditarik beberapa kesimpulan sebagai berikut:

- **Keberhasilan Implementasi Kriptografi Terdistribusi:** Sistem *Password Manager* berbasis CLI (*Command Line Interface*) berhasil dibangun dengan mengintegrasikan metode skema pembagian data rahasia *Shamir's Secret Sharing* threshold (2,3) dan algoritma enkripsi simetris AES-128-GCM. Sistem terbukti mampu memecah *Master Key* menjadi tiga komponen *share* (*Local Share*, *Server Share*, dan *Recovery Share*) secara matematis dan presisi.
- **Pemenuhan Prinsip Keamanan *Zero-Knowledge*:**  
Pengujian pada basis data server (SQLite) membuktikan bahwa prinsip *Zero-Knowledge Architecture* telah terpenuhi secara sempurna. Server hanya menyimpan data kredensial dalam bentuk objek biner acak (BLOB) terenkripsi serta satu pecahan kunci (*Server Share*). Server sama sekali tidak memiliki akses terhadap *Master Key* utuh maupun *Local Share* pengguna, sehingga data kata sandi tetap aman dari potensi kebocoran data di sisi server.
- **Keandalan Mekanisme Akses dan Pemulihan Darurat:**
  - Mode Normal berhasil berjalan secara otomatis dengan merekonstruksi *Master Key* melalui kombinasi *Local Share* (yang dilindungi oleh kunci KDF berbasis *Master Password* pengguna) dan *Server Share* dari database.
  - Mode Backup terbukti andal sebagai jalur pemulihan darurat ketika server disimulasikan mati (*down*). Dengan menggabungkan *Local Share* dan *Recovery Share* yang dimasukkan pengguna secara manual, sistem dapat mendekripsi *Backup Vault* lokal secara *read-only* demi mencegah inkonsistensi data.

- Efektivitas Fitur Bonus Kriptografi Visual: Fitur bonus berupa implementasi Kriptografi Visual skema  $(2,2)$  pada *Recovery Share* terbukti berjalan sukses. Sistem mampu mengkonversi string *Recovery Share* menjadi sebuah QR Code utuh, memecahnya menjadi dua citra *share* berbasis *noise* acak yang tidak dapat dipindai secara terpisah, dan memulihkannya kembali menjadi QR Code asli yang valid ketika kedua citra tersebut ditumpuk atau digabungkan (*combined.png*).

### 3.2. Saran

Meskipun sistem telah memenuhi seluruh spesifikasi utama tugas dan kasus uji yang dipersyaratkan, terdapat beberapa poin pengembangan yang disarankan untuk meningkatkan aspek keamanan, skalabilitas, dan kenyamanan pengguna di masa mendatang:

- Peningkatan Algoritma Enkripsi ke AES-256-GCM: Sistem saat ini menggunakan AES-128-GCM sesuai dengan spesifikasi minimum tugas. Untuk menghadapi ancaman komputasi di masa depan (*future-proofing*), disarankan untuk meningkatkan panjang kunci enkripsi menjadi AES-256-GCM yang menawarkan ruang kunci jauh lebih besar dan perlindungan yang lebih kuat terhadap serangan *brute-force*.
- Implementasi Protokol Komunikasi Aman (TLS/HTTPS): Pada pengembangan selanjutnya, komunikasi data antara aplikasi *client* dan *server* wajib diamankan menggunakan protokol pengiriman pesan yang terenkripsi seperti TLS/HTTPS atau *gRPC* dengan sistem autentikasi mutual (*mTLS*). Hal ini krusial untuk mencegah serangan *Man-in-the-Middle* (MitM) saat pemindahan *Server Share* dan *Encrypted Vault*.
- Pengembangan Antarmuka Pengguna (GUI/Ekstensi Browser): Format aplikasi berbasis CLI kurang ramah bagi pengguna awam. Pengembangan aplikasi ke bentuk GUI (Graphical User Interface) atau berupa Ekstensi Browser (seperti Bitwarden atau 1Password) akan meningkatkan retensi pengguna serta memudahkan fitur *auto-fill* kata sandi pada situs web secara langsung.

- Otomatisasi Sinkronisasi Vault (Resolusi Konflik Data): Mekanisme *Backup Vault* saat ini bersifat *read-only* untuk menghindari konflik data dengan server. Disarankan untuk menambahkan fitur sinkronisasi dua arah otomatis dengan resolusi konflik berbasis stempel waktu (*timestamp*) atau algoritma CRDT (*Conflict-free Replicated Data Type*). Sehingga, jika pengguna terpaksa melakukan perubahan data saat luring (offline), data dapat digabungkan secara aman saat server kembali online.
- Peningkatan Kualitas Citra Kriptografi Visual: Citra QR Code hasil rekonstruksi kriptografi visual (combined.png) terkadang memiliki penurunan kontras atau distorsi visual akibat tumpukan piksel *noise*. Disarankan untuk menggunakan metode *Kriptografi Visual Berwarna* atau teknik penyesuaian kontras citra tingkat lanjut agar hasil *scan* QR Code melalui kamera perangkat fisik menjadi lebih cepat dan responsif.

### 3.3. Lampiran

Lampiran berikut disajikan sebagai pelengkap dalam memenuhi spesifikasi yang telah ditetapkan.

- Pranala Repositori : [https://github.com/LeonArif/Tugas4\\_DZManager](https://github.com/LeonArif/Tugas4_DZManager)
- Pranala Video Demo :  Video Demo.mp4
- Pembagian Tugas

| Nama                   | NIM      | Tugas                                                                                                                                                                                                                 |
|------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Khairunisa Azizah      | 18223117 | <ul style="list-style-type: none"> <li>● Membuat penyimpanan server/database SQLite</li> <li>● Membuat HTTP server/backend utama</li> <li>● Membuat alur CLI untuk client</li> <li>● Menulis Laporan</li> </ul>       |
| Leonard Arif Sutiono   | 18223120 | <ul style="list-style-type: none"> <li>● Membuat fitur <i>recovery</i></li> <li>● Membuat fitur <i>edit</i> dan <i>delete</i></li> <li>● Membuat fitur bonus visual kriptografi</li> <li>● Menulis Laporan</li> </ul> |
| Nakeisha Valya Shakila | 18223133 | <ul style="list-style-type: none"> <li>● Membuat algoritma pembangkit password dan random data</li> <li>● Membuat algoritma derivasi kunci dan</li> </ul>                                                             |



|  |  |                                                                                                                                                                  |
|--|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |  | <p>proteksi <i>client</i></p> <ul style="list-style-type: none"><li>• Membuat algoritma enkripsi/dekripsi dan secret sharing</li><li>• Menulis Laporan</li></ul> |
|--|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------|